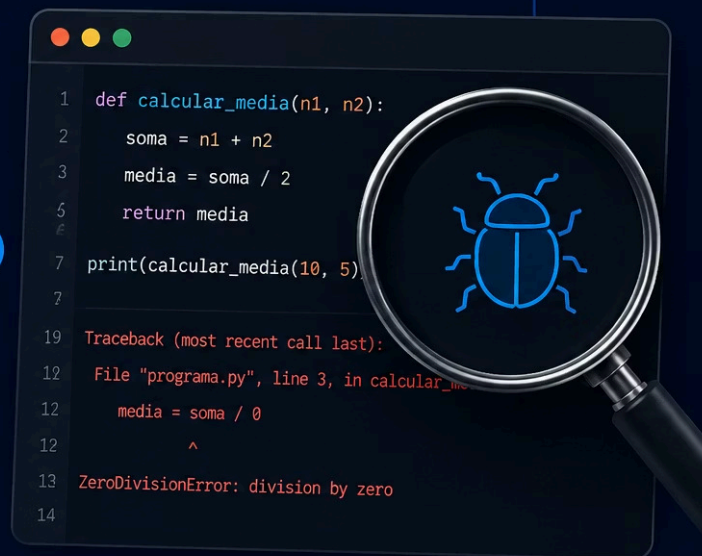




Tipos de Erros: o primeiro passo para depurar

Antes de corrigir um **bug**,
precisamos entender o tipo do **erro**.



Nem todo erro é igual

Assim como um detetive investiga pistas diferentes, cada tipo de erro exige uma abordagem distinta. Reconhecer o tipo é o primeiro passo para resolver o problema.

🚫 Impede de rodar

O código nem começa a executar.

💥 Para no meio

O programa inicia, mas trava durante a execução.

🤔 Funciona errado

Roda sem avisar, mas entrega resultado incorreto.

🔍 **Lembre-se:** Você não resolve um mistério sem entender a pista.



● ERRO #1

Erro de Sintaxe

O código **quebra as regras da linguagem**. É como escrever uma frase sem fechar a pontuação: o Python simplesmente não entende o comando.

✗ Código com erro

```
print("Olá"
```

✓ Código corrigido

```
print("Olá")
```

💬 Mensagem do Python

```
SyntaxError: unexpected EOF  
while parsing
```

O Python apontou exatamente onde a frase ficou incompleta. A mensagem de erro é sua primeira pista!

Como reconhecer um Erro de Sintaxe?

O Python é rigoroso. Pequenos deslizes quebram o código antes mesmo de rodar.

(

Parênteses abertos

```
print("Olá"
```

66

Aspas não fechadas

```
nome = "Ana
```

:

Dois-pontos esquecidos

```
if x > 0
```

≡

Indentação incorreta

```
print("oi")
```

✎

Palavra digitada errado

```
prnt("Olá")
```

Erro de Execução (Runtime)

O programa **começa a rodar**, mas encontra um problema no caminho. O código parecia certo — mas algo deu errado durante a execução.

✗ Exemplo clássico

```
x = 10  
y = 0  
resultado = x / y
```

💬 Mensagem do Python

```
ZeroDivisionError:  
division by zero
```

Outros erros comuns

→ **NameError**

Variável que não existe

→ **TypeError**

Operação com tipos incompatíveis

→ **IndexError**

Índice fora do limite da lista

Como reconhecer um Erro de Execução?

⚡ Programa para de repente

A execução é interrompida antes do fim.

📋 Terminal mostra "Traceback"

O Python exibe o caminho do erro com o nome da exceção.

🎯 Erro depende da entrada

Funciona com alguns dados, mas falha com outros.



Sinais de alerta no terminal

Observe como os sintomas aparecem juntos: a execução para, o terminal destaca o traceback e o comportamento muda conforme a entrada.

● ERRO #3

Erro de Lógica

O programa **roda sem nenhuma mensagem de erro**, mas entrega um resultado errado. O computador fez exatamente o que você mandou... não o que você queria.

✗ Código com erro lógico

```
nota1 = 8  
nota2 = 6  
media = nota1 + nota2 / 2  
print(media) # Resultado: 11.0
```

Como reconhecer um Erro de Lógica?

Este é o mais sorrateiro: **não há mensagem de erro**. O programa roda, mas o resultado não faz sentido.



Resultado estranho

O valor calculado parece errado ou inesperado.

$+ = \times$

Fórmula errada

A ordem das operações ou a expressão matemática está incorreta.



Condição invertida

Um `if` que deveria ser verdadeiro sempre retorna falso.



Repetições incorretas

Um `for` ou `while` que executa vezes demais ou de menos.



O programa rodou... mas o resultado faz sentido?

Comparando os Tipos de Erro

Cada erro tem sua "assinatura". Aprenda a identificá-la antes de agir.

Tipo de Erro	O programa roda?	Há mensagem?	Exemplo
 Sintaxe	Não	Sim, <code>SyntaxError</code>	<code>print("Olá"</code>
 Runtime	Começa e quebra	Sim, exceção	<code>10 / 0</code>
 Lógica	Sim	Nem sempre	<code>media = a + b / 2</code>

Sintaxe

Regras quebradas

Runtime

Falha na execução

Lógica

Resultado incorreto

Agora é sua vez: que tipo de erro é esse?

No **Kahoot!**, você vai analisar exemplos reais e classificar cada bug. Lembre-se das pistas de cada tipo!

● Sintaxe

O código não roda. Procure por parênteses, aspas e dois-pontos.

● Runtime

O programa para no meio. Procure por Traceback e exceções.

● Lógica

Roda, mas o resultado está errado. Questione a lógica do código.

✅ 🏆 Na Debug Arena, cada erro é uma pista. O desafio é **investigar antes de corrigir**.

